

GDScript style guide

This style guide lists conventions to write elegant GDScript. The goal is to encourage writing clean, readable code and promote consistency across projects, discussions, and tutorials. Hopefully, this will also support the development of auto-formatting tools.

Since GDScript is close to Python, this guide is inspired by Python's [PEP 8](#) programming style guide.

Style guides aren't meant as hard rulebooks. At times, you may not be able to apply some of the guidelines below. When that happens, use your best judgment, and ask fellow developers for insights.

In general, keeping your code consistent in your projects and within your team is more important than following this guide to a tee.

📌 Note

Godot's built-in script editor uses a lot of these conventions by default. Let it help you.

Here is a complete class example based on these guidelines:

```

class_name StateMachine
extends Node
## Hierarchical State machine for the player.
##
## Initializes states and delegates engine callbacks ([method
Node._physics_process],
## [method Node._unhandled_input]) to the state.

signal state_changed(previous, new)

@export var initial_state: Node
var is_active = true:
    set = set_is_active

@onready var _state = initial_state:
    set = set_state
@onready var _state_name = _state.name

func _init():
    add_to_group("state_machine")

func _enter_tree():
    print("this happens before the ready method!")

func _ready():
    state_changed.connect(_on_state_changed)
    _state.enter()

func _unhandled_input(event):
    _state.unhandled_input(event)

func _physics_process(delta):
    _state.physics_process(delta)

func transition_to(target_state_path, msg={}):
    if not has_node(target_state_path):
        return

    var target_state = get_node(target_state_path)
    assert(target_state.is_composite == false)

```

```

_state.exit()
self._state = target_state
_state.enter(msg)
Events.player_state_changed.emit(_state.name)

func set_is_active(value):
    is_active = value
    set_physics_process(value)
    set_process_unhandled_input(value)
    set_block_signals(not value)

func set_state(value):
    _state = value
    _state_name = _state.name

func _on_state_changed(previous, new):
    print("state changed")
    state_changed.emit()

class State:
    var foo = 0

    func _init():
        print("Hello!")

```

Formatting

Encoding and special characters

- Use line feed (**LF**) characters to break lines, not CRLF or CR. *(editor default)*
- Use one line feed character at the end of each file. *(editor default)*
- Use **UTF-8** encoding without a [byte order mark](#) . *(editor default)*
- Use **Tabs** instead of spaces for indentation. *(editor default)*

Indentation

Each indent level should be one greater than the block containing

Good:

```
for i in range(10):  
    print("hello")
```

Bad:

```
for i in range(10):  
    print("hello")  
  
for i in range(10):  
    print("hello")
```

Use 2 indent levels to distinguish continuation lines from regular code blocks.

Good:

```
effect.interpolate_property(sprite, "transform/scale",  
    sprite.get_scale(), Vector2(2.0, 2.0), 0.3,  
    Tween.TRANS_QUAD, Tween.EASE_OUT)
```

Bad:

```
effect.interpolate_property(sprite, "transform/scale",  
    sprite.get_scale(), Vector2(2.0, 2.0), 0.3,  
    Tween.TRANS_QUAD, Tween.EASE_OUT)
```

Exceptions to this rule are arrays, dictionaries, and enums. Use a single indentation level to distinguish continuation lines:

Good:

```
var party = [  
    "Godot",  
    "Godette",  
    "Steve",  
]  
  
var character_dict = {  
    "Name": "Bob",  
    "Age": 27,  
    "Job": "Mechanic",  
}  
  
enum Tile {  
    BRICK,  
    FLOOR,  
    SPIKE,  
    TELEPORT,  
}
```

Bad:

```
var party = [  
    "Godot",  
    "Godette",  
    "Steve",  
]  
  
var character_dict = {  
    "Name": "Bob",  
    "Age": 27,  
    "Job": "Mechanic",  
}  
  
enum Tile {  
    BRICK,  
    FLOOR,  
    SPIKE,  
    TELEPORT,  
}
```

Trailing comma

Use a trailing comma on the last line in arrays, dictionaries, and enums. This results in easier refactoring and better diffs in version control as the last line doesn't need to be modified when adding new elements.

Good:

```
var array = [  
    1,  
    2,  
    3,  
]
```

Bad:

```
var array = [  
    1,  
    2,  
    3  
]
```

Trailing commas are unnecessary in single-line lists, so don't add them in this case.

Good:

```
var array = [1, 2, 3]
```

Bad:

```
var array = [1, 2, 3,]
```

Blank lines

Surround functions and class definitions with two blank lines:

```
func heal(amount):
    health += amount
    health = min(health, max_health)
    health_changed.emit(health)

func take_damage(amount, effect=null):
    health -= amount
    health = max(0, health)
    health_changed.emit(health)
```

Use one blank line inside functions to separate logical sections.

! Note

We use a single line between classes and function definitions in the class reference and in short code snippets in this documentation.

Line length

Keep individual lines of code under 100 characters.

If you can, try to keep lines under 80 characters. This helps to read the code on small displays and with two scripts opened side-by-side in an external text editor. For example, when looking at a differential revision.

One statement per line

Avoid combining multiple statements on a single line, including conditional statements, to adhere to the GDScript style guidelines for readability.

Good:

```
if position.x > width:
    position.x = 0

if flag:
    print("flagged")
```

Bad:

```
if position.x > width: position.x = 0

if flag: print("flagged")
```

The only exception to that rule is the ternary operator:

```
next_state = "idle" if is_on_floor() else "fall"
```

Format multiline statements for readability

When you have particularly long `if` statements or nested ternary expressions, wrapping them over multiple lines improves readability. Since continuation lines are still part of the same expression, 2 indent levels should be used instead of one.

GDScript allows wrapping statements using multiple lines using parentheses or backslashes. Parentheses are favored in this style guide since they make for easier refactoring. With backslashes, you have to ensure that the last line never contains a backslash at the end. With parentheses, you don't have to worry about the last line having a backslash at the end.

When wrapping a conditional expression over multiple lines, the `and` / `or` keywords should be placed at the beginning of the line continuation, not at the end of the previous line.

Good:

```

var angle_degrees = 135
var quadrant = (
    "northeast" if angle_degrees <= 90
    else "southeast" if angle_degrees <= 180
    else "southwest" if angle_degrees <= 270
    else "northwest"
)

var position = Vector2(250, 350)
if (
    position.x > 200 and position.x < 400
    and position.y > 300 and position.y < 400
):
    pass

```

Bad:

```

var angle_degrees = 135
var quadrant = "northeast" if angle_degrees <= 90 else "southeast" if
angle_degrees <= 180 else "southwest" if angle_degrees <= 270 else
"northwest"

var position = Vector2(250, 350)
if position.x > 200 and position.x < 400 and position.y > 300 and
position.y < 400:
    pass

```

Avoid unnecessary parentheses

Avoid parentheses in expressions and conditional statements. Unless necessary for order of operations or wrapping over multiple lines, they only reduce readability.

Good:

```

if is_colliding():
    queue_free()

```

Bad:

```
if (is_colliding()):  
    queue_free()
```

Boolean operators

Prefer the plain English versions of boolean operators, as they are the most accessible:

- Use **and** instead of **&&** .
- Use **or** instead of **||** .
- Use **not** instead of **!** .

You may also use parentheses around boolean operators to clear any ambiguity. This can make long expressions easier to read.

Good:

```
if (foo and bar) or not baz:  
    print("condition is true")
```

Bad:

```
if foo && bar || !baz:  
    print("condition is true")
```

Comment spacing

Regular comments (**#**) and documentation comments (**##**) should start with a space, but not code that you comment out. Additionally, code region comments (**#region** / **#endregion**) must follow that precise syntax, so they should not start with a space.

Using a space for regular and documentation comments helps differentiate text comments from disabled code.

Good:

```
# This is a comment.  
#print("This is disabled code")
```

Bad:

```
#This is a comment.  
# print("This is disabled code")
```

! Note

In the script editor, to toggle commenting of the selected code, press `Ctrl + K`. This feature adds/removes a single `#` sign before any code on the selected lines.

Prefer writing comments on their own line as opposed to inline comments (comments written on the same line as code). Inline comments are best used for short comments, typically a few words at most:

Good:

```
# This is a long comment that would make the line below too long if written  
inline.  
print("Example") # Short comment.
```

Bad:

```
print("Example") # This is a long comment that would make this line too  
long if written inline.
```

Whitespace

Always use one space around operators and after commas. Also, avoid extra spaces in dictionary references and function calls. One exception to this is for single-line dictionary declarations, where a space should be added after the `{` before the closing brace. This makes the dictionary easier to visually distinguish from an array, as the `[]` characters look close to `{}` with most fonts.

Good:

```
position.x = 5
position.y = target_position.y + 10
dict["key"] = 5
my_array = [4, 5, 6]
my_dictionary = { key = "value" }
print("foo")
```

Bad:

```
position.x=5
position.y = mpos.y+10
dict ["key"] = 5
myarray = [4,5,6]
my_dictionary = {key = "value"}
print ("foo")
```

Don't use spaces to align expressions vertically:

```
x          = 100
y          = 100
velocity = 500
```

Quotes

Use double quotes unless single quotes make it possible to escape fewer characters in a given string. See the examples below:

```
# Normal string.
print("hello world")

# Use double quotes as usual to avoid escapes.
print("hello 'world'")

# Use single quotes as an exception to the rule to avoid escapes.
print('hello "world"')

# Both quote styles would require 2 escapes; prefer double quotes if it's a
tie.
print("'hello' \"world\"")
```

Numbers

Don't omit the leading or trailing zero in floating-point numbers. Otherwise, this makes them less readable and harder to distinguish from integers at a glance.

Good:

```
var float_number = 0.234
var other_float_number = 13.0
```

Bad:

```
var float_number = .234
var other_float_number = 13.
```

Use lowercase for letters in hexadecimal numbers, as their lower height makes the number easier to read.

Good:

```
var hex_number = 0xfb8c0b
```

  en  stable ▼

Bad:

```
var hex_number = 0xFB8C0B
```

Take advantage of GDScript's underscores in literals to make large numbers more readable.

Good:

```
var large_number = 1_234_567_890
var large_hex_number = 0xffff_f8f8_0000
var large_bin_number = 0b1101_0010_1010
# Numbers lower than 1000000 generally don't need separators.
var small_number = 12345
```

Bad:

```
var large_number = 1234567890
var large_hex_number = 0xffffffff8f80000
var large_bin_number = 0b11010010101010
# Numbers lower than 1000000 generally don't need separators.
var small_number = 12_345
```

Naming conventions

These naming conventions follow the Godot Engine style. Breaking these will make your code clash with the built-in naming conventions, leading to inconsistent code. As a summary table:

Type	Convention	Example
File names	snake_case	yaml_parser.gd
Class names	PascalCase	class_name YAMLParser
Node names	PascalCase	Camera3D , Player
Functions	snake_case	func load_level():
Variables	snake_case	var particle_effect
Signals	snake_case	signal door_opened

Type	Convention	Example
Constants	CONSTANT_CASE	<code>const MAX_SPEED = 200</code>
Enum names	PascalCase	<code>enum Element</code>
Enum members	CONSTANT_CASE	<code>{EARTH, WATER, AIR, FIRE}</code>

File names

Use snake_case for file names. For named classes, convert the PascalCase class name to snake_case:

```
# This file should be saved as `weapon.gd`.
class_name Weapon
extends Node
```

```
# This file should be saved as `yaml_parser.gd`.
class_name YAMLParser
extends Object
```

This is consistent with how C++ files are named in Godot's source code. This also avoids case sensitivity issues that can crop up when exporting a project from Windows to other platforms.

Classes and nodes

Use PascalCase for class and node names:

```
extends CharacterBody3D
```

Also use PascalCase when loading a class into a constant or a variable:

```
const Weapon = preload("res://weapon.gd")
```

Functions and variables

Use snake_case to name functions and variables:

```
var particle_effect  
func load_level():
```

Prepend a single underscore (_) to virtual methods functions the user must override, private functions, and private variables:

```
var _counter = 0  
func _recalculate_path():
```

Signals

Use the past tense to name signals:

```
signal door_opened  
signal score_changed
```

Constants and enums

Write constants with CONSTANT_CASE, that is to say in all caps with an underscore (_) to separate words:

```
const MAX_SPEED = 200
```

Use PascalCase for enum *names* and keep them singular, as they are constants. Use CONSTANT_CASE for their members, as they are constants:

```
enum Element {  
    EARTH,  
    WATER,  
    AIR,  
    FIRE,  
}
```

Write enums with each item on its own line. This allows adding documentation comments above each item more easily, and also makes for cleaner diffs in version control when items are added or removed.

Good:

```
enum Element {  
    EARTH,  
    WATER,  
    AIR,  
    FIRE,  
}
```

Bad:

```
enum Element { EARTH, WATER, AIR, FIRE }
```

Code order

This section focuses on code order. For formatting, see [Formatting](#). For naming conventions, see [Naming conventions](#).

We suggest to organize GDScript code this way:

```
01. @tool, @icon, @static_unload
02. class_name
03. extends
04. ## doc comment

05. signals
06. enums
07. constants
08. static variables
09. @export variables
10. remaining regular variables
11. @onready variables

12. _static_init()
13. remaining static methods
14. overridden built-in virtual methods:
    1. _init()
    2. _enter_tree()
    3. _ready()
    4. _process()
    5. _physics_process()
    6. remaining virtual methods
15. overridden custom methods
16. remaining methods
17. inner classes
```

And put the class methods and variables in the following order depending on their access modifiers:

```
1. public
2. private
```

We optimized the order to make it easy to read the code from top to bottom, to help developers reading the code for the first time understand how it works, and to avoid errors linked to the order of variable declarations.

This code order follows four rules of thumb:

1. Properties and signals come first, followed by methods.
2. Public comes before private.

3. Virtual callbacks come before the class's interface.
4. The object's construction and initialization functions, `_init` and `_ready`, come before functions that modify the object at runtime.

Class declaration

If the code is meant to run in the editor, place the `@tool` annotation on the first line of the script.

Follow with the optional `@icon` then the `class_name` if necessary. You can turn a GDScript file into a global type in your project using `class_name`. For more information, see [Registering named classes](#). If the class is meant to be an [abstract class](#), add `@abstract` before the `class_name` keyword.

Then, add the `extends` keyword if the class extends a built-in type.

Following that, you should have the class's optional [documentation comments](#). You can use that to explain the role of your class to your teammates, how it works, and how other developers should use it, for example.

```
@abstract
class_name MyNode
extends Node
## A brief description of the class's role and functionality.
##
## The description of the script, what it can do,
## and any further detail.
```

For inner classes, use single-line declarations:

```
## A brief description of the class's role and functionality.
##
## The description of the script, what it can do,
## and any further detail.
@abstract class MyNode extends Node:
    pass
```

Signals and properties

Write signal declarations, followed by properties, that is to say, member variables, after the docstring.

Enums should come after signals, as you can use them as export hints for other properties.

Then, write constants, exported variables, public, private, and onready variables, in that order.

```
signal player_spawned(position)

enum Job {
    KNIGHT,
    WIZARD,
    ROGUE,
    HEALER,
    SHAMAN,
}

const MAX_LIVES = 3

@export var job: Job = Job.KNIGHT
@export var max_health = 50
@export var attack = 5

var health = max_health:
    set(new_health):
        health = new_health

var _speed = 300.0

@onready var sword = get_node("Sword")
@onready var gun = get_node("Gun")
```

Note

GScript evaluates `@onready` variables right before the `_ready` callback. You can use that to cache node dependencies, that is to say, to get child  `en`  `stable`  that your class relies on. This is what the example above shows.

Member variables

Don't declare member variables if they are only used locally in a method, as it makes the code more difficult to follow. Instead, declare them as local variables in the method's body.

Local variables

Declare local variables as close as possible to their first use. This makes it easier to follow the code, without having to scroll too much to find where the variable was declared.

Methods and static functions

After the class's properties come the methods.

Start with the `_init()` callback method, that the engine will call upon creating the object in memory. Follow with the `_ready()` callback, that Godot calls when it adds a node to the scene tree.

These functions should come first because they show how the object is initialized.

Other built-in virtual callbacks, like `_unhandled_input()` and `_physics_process`, should come next. These control the object's main loop and interactions with the game engine.

The rest of the class's interface, public and private methods, come after that, in that order.

```

func _init():
    add_to_group("state_machine")

func _ready():
    state_changed.connect(_on_state_changed)
    _state.enter()

func _unhandled_input(event):
    _state.unhandled_input(event)

func transition_to(target_state_path, msg={}):
    if not has_node(target_state_path):
        return

    var target_state = get_node(target_state_path)
    assert(target_state.is_composite == false)

    _state.exit()
    self._state = target_state
    _state.enter(msg)
    Events.player_state_changed.emit(_state.name)

func _on_state_changed(previous, new):
    print("state changed")
    state_changed.emit()

```

Static typing

GDScript supports [optional static typing](#).

Declared types

To declare a variable's type, use `<variable>: <type> :`

```
var health: int = 0
```

To declare the return type of a function, use `-> <type> :`

```
func heal(amount: int) -> void:
```

Inferred types

In most cases, you can let the compiler infer the type using `:=`. Prefer `:=` when the type is written on the same line as the assignment, otherwise prefer writing the type explicitly.

Good:

```
# The type can be int or float, and thus should be stated explicitly.
var health: int = 0

# The type is clearly inferred as Vector3.
var direction := Vector3(1, 2, 3)
```

Include the type hint when the type is ambiguous, and omit the type hint when it's redundant.

Bad:

```
# Typed as int, but it could be that float was intended.
var health := 0

# The type hint has redundant information.
var direction: Vector3 = Vector3(1, 2, 3)

# What type is this? It's not immediately clear to the reader, so it's bad.
var value := complex_function()
```

In some cases, the type must be stated explicitly, otherwise the behavior will not be as expected because the compiler will only be able to use the function's return type. For example, `get_node()` cannot infer a type unless the scene or file  [en](#)  [stable](#)  memory. In this case, you should set the type explicitly.

Good:

```
@onready var health_bar: ProgressBar = get_node("UI/LifeBar")
```

Bad:

```
# The compiler can't infer the exact type and will use Node
# instead of ProgressBar.
@onready var health_bar := get_node("UI/LifeBar")
```

Alternatively, you can use the `as` keyword to cast the return type, and that type will be used to infer the type of the var.

```
@onready var health_bar := get_node("UI/LifeBar") as ProgressBar
# health_bar will be typed as ProgressBar
```

Note

This option is considered more [type-safe](#) than type hints, but also less null-safe as it silently casts the variable to `null` in case of a type mismatch at runtime, without an error/warning.

[< Previous](#)[Next >](#)

User-contributed notes

Please read the [User-contributed notes policy](#) before submitting a comment.

0 comments

Write

Preview

Aa

Sign in to comment

M↓

Sign in with GitHub